

EAPD711 PART 1

Uafhulufhedzea Rambuda

ST10127213

Number of words: 1315

Contents

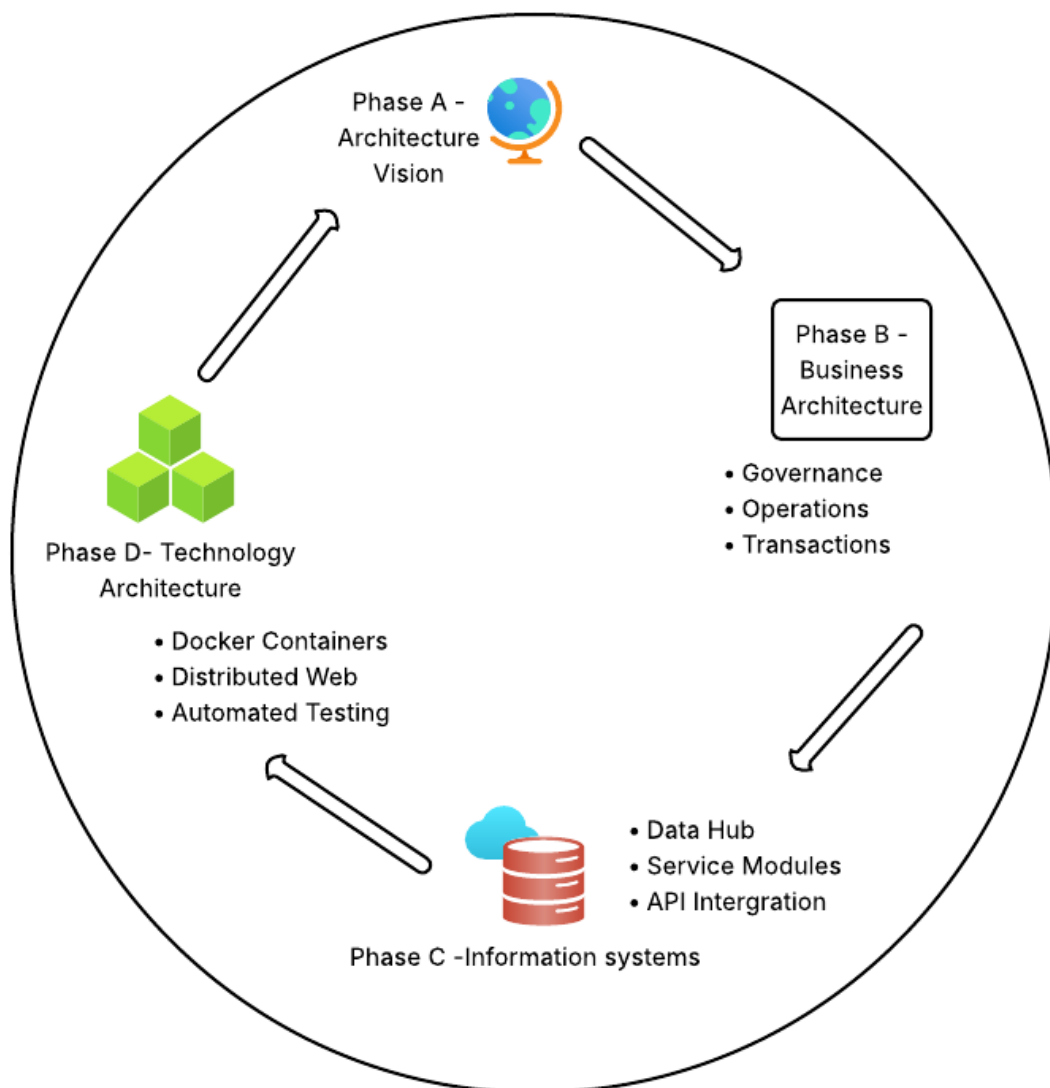
Q1.....	2
Q2.....	4
Q3.....	6
References	7

Q1

1. Architectural Framework Strategy.

For a project as complex as the **Global Logistics Management System (GLMS)**, simply jumping into coding isn't enough. TechMove is currently dealing with a massive "fragmented legacy environment"—basically, a mess of spreadsheets and emails that don't talk to each other. To fix this, I've chosen **TOGAF (The Open Group Architecture Framework)** as our primary strategy.

The reason TOGAF works here is because of its **Architecture Development Method (ADM)**. It's a step-by-step cycle that helps a company move from a "disorganized" state to a "modern" one. Instead of just building a website, TOGAF ensures that the new system actually solves the business problems TechMove is facing, like lost contracts or expired driver schedules.



Phase A: Architecture Vision

First, we have to look at the "big picture." Right now, TechMove's data is scattered across different offices. The vision for GLMS is to create a single, centralized web platform that acts as the "source of truth." In this phase, we sit down with the stakeholders—like the logistics managers and the IT team—to agree on what the system *must* do. We're moving from "manual and risky" to "automated and secure." This vision acts as our North Star throughout the entire development process.

Phase B: Business Architecture

Next, we have to figure out how TechMove actually operates day-to-day. We can't build a contract hub if we don't understand how a contract moves from a draft to being signed. In Phase B, we map out these workflows. For example, we define the "Contract Lifecycle": how it starts as a **Draft**, becomes **Active**, and eventually reaches **Expiry**. By defining these business rules now, we ensure the software's logic (like blocking a service request if a contract is expired) actually fits how the logistics world works.

Phase C: Information Systems Architecture

This is where we get into the "guts" of the system, divided into two parts:

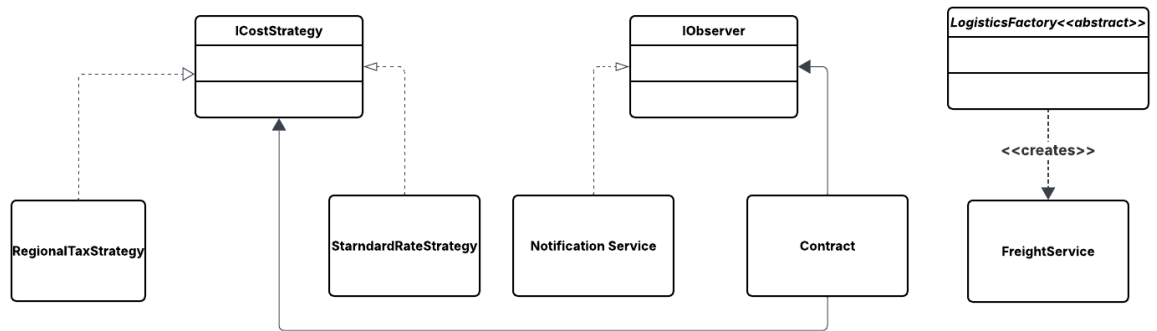
- **Data Architecture:** We need to design how we store things like international freight rates and driver info. We're basically cleaning up the "spreadsheet mess" by creating a solid database schema that handles multiple currencies and audit logs.
- **Application Architecture:** We're moving away from a "monolith" (one big, clunky program) toward a **Service-Oriented Architecture (SOA)**. This means we break the GLMS into smaller, independent parts—like a specific service for "Currency Conversion" and another for "Contract Management"—so they are easier to update later.

Phase D: Technology Architecture

Finally, we look at the actual tech stack. For the GLMS, we're going to use **Docker containers**. This is a huge deal for TechMove because it means the software will run exactly the same way on a developer's laptop as it does on the live server. We also plan out how the system will talk to **External APIs** to get real-time currency exchange rates. This ensures that when a service invoice is generated, the financial data is 100% accurate across different countries.

Q2

2. Design Pattern Selection & Implementation



For the GLMS to handle the complexity of international logistics. We need a design that is flexible enough to change without breaking. I've selected three **Gang of Four (GoF)** patterns that specifically solve TechMove's biggest pain points.

I . Strategy Pattern (Financial Flexibility)

Since TechMove deals with different countries, we can't have one single way to calculate costs. One country might have a specific VAT, while another uses a flat freight tax.

- **The Problem:** Hard-coding these rules makes the system a nightmare to update.
- **The Human Solution:** I'm using the **Strategy Pattern**. We define a general rule for calculating costs, and then we create specific "strategies" for USD, EUR, or ZAR. When the system generates an invoice, it just "plugs in" the right strategy for that specific contract. It keeps the core code clean and ensures we stay financially compliant.

II. Observer Pattern (Automated Compliance)

The board is worried about "expired contracts" and "manual tracking." The **Observer Pattern** is the answer to this.

- **The Problem:** In the legacy system, a manager had to manually check if a contract was ending. If they forgot, things won't go well.
- **The Human Solution:** I'm setting up the **Contract** class as a "Subject." Then, I have "Observers" like a **NotificationService** and an **AlertDashboard**. The moment a contract status changes (like moving from 'Active' to 'Expired'), the contract "shouts" out to its observers. The observers then automatically send emails or update the dashboard. It moves the responsibility from the human to the system.

III. Factory Method Pattern (Scalable Service Creation)

Logistics isn't just one thing—it's road freight, air freight, and ocean freight. Each has different rules and data.

- **The Problem:** Using if/else statements to create these different services makes the code messy and hard to scale.
- **The Human Solution:** I'm implementing the **Factory Method**. Instead of the main system trying to build a freight object itself, it asks a ServiceFactory to do it. The Factory looks at the request and "manufactures" the right object (like a RoadFreight object). If TechMove adds drone deliveries later, we just add a new "factory line" without touching the existing air or road code.

Q3

3. Standards and Scalability Strategy

As TechMove Logistics prepares for the massive influx of over **50,000 new international contracts**, the Global Logistics Management System (GLMS) must move beyond basic functionality. To ensure the system is enterprise-ready, we are establishing strict standards for **Availability** and **Interoperability**, backed by a modern **Scalability Strategy**.

Availability and Reliability Standards

In the world of global logistics, downtime equals lost revenue and stranded freight. To address this, we are setting an **Availability standard of 99.9% (Three Nines)**. We achieve this by leveraging the **Docker containerization** mentioned in our TOGAF roadmap. By deploying the GLMS in a containerized environment, we can implement "Health Checks." If a specific service—like the Contract Hub—fails, the system automatically detects the failure and spins up a fresh container in seconds. This "self-healing" architecture ensures that logistics managers in different time zones always have access to their data, fulfilling the requirement for a reliable, distributed web platform.

Interoperability and API Integration

The background information highlighted the need for "automated international currency conversion." This requires the GLMS to "talk" to external financial systems seamlessly. We are adopting the **RESTful API standard** using **JSON** for all data exchange. This ensures **Interoperability** between TechMove's internal C# services and third-party banking APIs. By using these universal standards, the GLMS can pull real-time exchange rates and tax data without manual entry, finally eliminating the fragmented email and spreadsheet processes used in the legacy system.

Scalability Strategy for 50,000+ Contracts

Handling 50,000+ contracts requires a shift from vertical scaling (buying more powerful servers) to **Horizontal Scaling**. Because we have designed the GLMS using a **Service-Oriented Architecture (SOA)**, we can scale individual parts of the system independently.

- **Load Balancing:** As traffic grows, we will use a Load Balancer to distribute requests across multiple instances of the GLMS.
- **Database Sharding:** To manage the massive volume of contract data, we can "shard" or split the database by region. This ensures that a query for a European contract doesn't get slowed down by thousands of records from the North American division.

This multi-layered approach ensures that as TechMove grows, the GLMS remains fast, responsive, and capable of handling global logistics at an enterprise scale.

References

The Open Group (2022) *TOGAF Standard, Version 10.0: An Introduction*. Available at: opengroup.org (Accessed: 13 April 2026).

Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (2023) *Design Patterns: Elements of Reusable Object-Oriented Software*. Digital Version. Available at: oreilly.com (Accessed: 13 April 2026).

Microsoft Learn (2024) *Cloud Design Patterns: Scalability and Availability*. Available at: microsoft.com (Accessed: 13 April 2026).

Docker Documentation (2025) *What is a Container?*. Available at: docker.com (Accessed: 13 April 2026).

IBM Education (2023) *What is Service-Oriented Architecture (SOA)?*. Available at: ibm.com (Accessed: 13 April 2026).